2.1.4 Calling the Mailer

Mailers are really just another way to render a view. Instead of rendering a view and sending out the HTTP protocol, they are just sending it out through the email protocols instead. Due to this, it makes sense to just have your controller tell the Mailer to send an email when a user is successfully created.

Setting this up is painfully simple.

First, let's create a simple `User` scaffold:

```
$ bin/rails generate scaffold user name email login
$ bin/rake db:migrate
```

Now that we have a user model to play with, we will just edit the `app/controllers/users_controller.rb` make it instruct the `UserMailer` to deliver an email to the newly created user by editing the create action and inserting a call to `UserMailer.welcome_email`right after the user is successfully saved.

Action Mailer is nicely integrated with Active Job so you can send emails outside of the request-response cycle, so the user doesn't have to wait on it:

```
class UsersController < ApplicationController
  # POST /users
  # POST /users.json
  def create
    @user = User.new(params[:user])

    respond_to do |format|
      if @user.save
        # Tell the UserMailer to send a welcome email after save
        UserMailer.welcome_email(@user).deliver_later

        format.html { redirect_to(@user, notice: 'User was
successfully created.') }
        format.json { render json: @user, status: :created, location:
@user }
      else
        format.html { render action: 'new' }
        format.json { render json: @user.errors, status:
:unprocessable_entity }
      end
    end
  end
end
```

Active Job's default behavior is to execute jobs ':inline'. So, you can use `deliver_later`now to send emails, and when you later decide to start sending them from a background job, you'll only need to set up Active Job to use a queueing backend (Sidekiq, Resque, etc).

If you want to send emails right away (from a cronjob for example) just call `deliver_now`:

```
class SendWeeklySummary
  def run
    User.find_each do |user|
      UserMailer.weekly_summary(user).deliver_now
    end
  end
end
```

The method `welcome_email` returns a `ActionMailer::MessageDelivery` object which can then just be told `deliver_now` or `deliver_later` to send itself out. The `ActionMailer::MessageDelivery`object is just a wrapper around a `Mail::Message`. If you want to inspect, alter or do anything else with